

大数据笔记

第1章 大数据概述

第2章 大数据处理架构Hadoop

第3章 分布式文件系统HDFS

第4章 分布式数据库HBase

第5章 NoSQL数据库

第6章 云数据库

第7章 MapReduce

第8章 Hadoop再探讨

第9章 数据仓库Hive

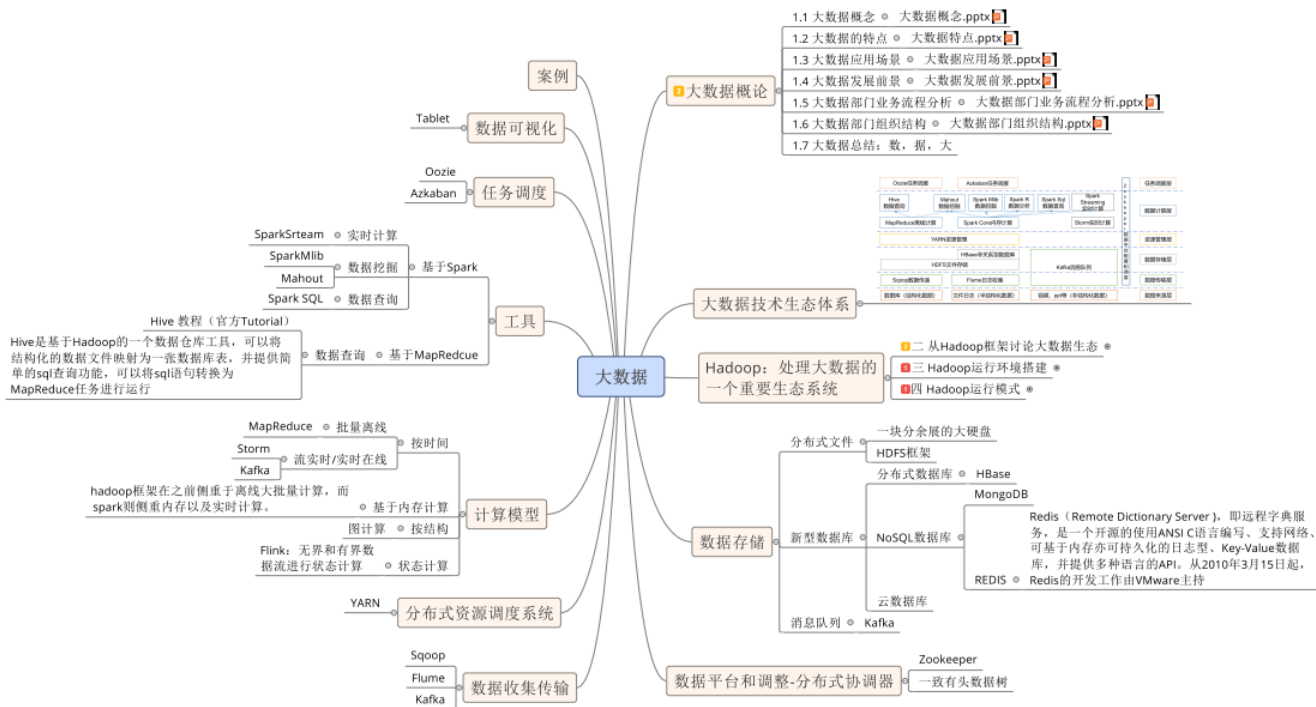
第10章 Spark

第11章 流计算

第12章 Flink

第13章 图计算

第16.4章 基于大数据的综合健康服务平台



第1章 大数据概述

信息化浪潮	发生时间	标志	解决问题	代表企业
第一次浪潮	1980年前后	个人计算机	信息处理	Intel、AMD、IBM、苹果、微软、联想、戴尔、惠普等
第二次浪潮	1995年前后	互联网	信息传输	雅虎、谷歌、阿里巴巴、百度、腾讯等
第三次浪潮	2010年前后	物联网、云计算和大数据	信息爆炸	Facebook、亚马逊、美团、今日头条、滴滴

数据也被称为“未来的石油”

数据产生方式的变革：

运营式系统阶段：数据库，数据产生是被动

用户原创内容阶段：主动

感知式系统阶段：物联网的发展

发展历程：萌芽期，成熟期，大规模应用期

主要解决：海量数据的**存储**和海量数据的**分析计算**（批处理，实时处理，流实时处理）问题

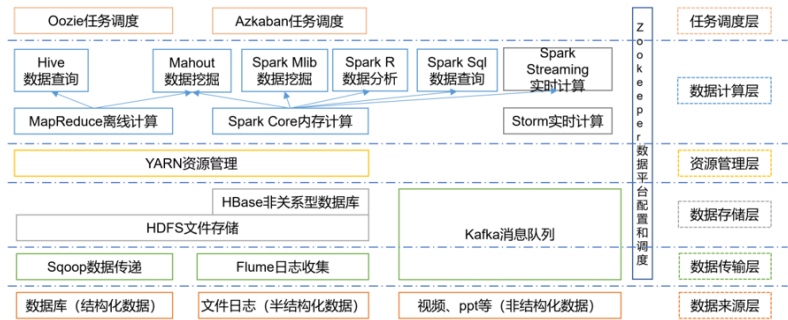
特点：4V 数据量大（volume），数据类型繁多(volume)，处理速度快（velocity),价值密度低（value）

范式：实验，理论，计算，数据

大数据完全颠覆了传统的思维方式：全样而非抽样,效率而非精确,相关而非因果
采集，存储，分析，应用

大数据关键技术：数据采集与预处理，数据存储和管理，数据处理和分析，数据安全和隐私保护
大数据计算模式及其代表产品

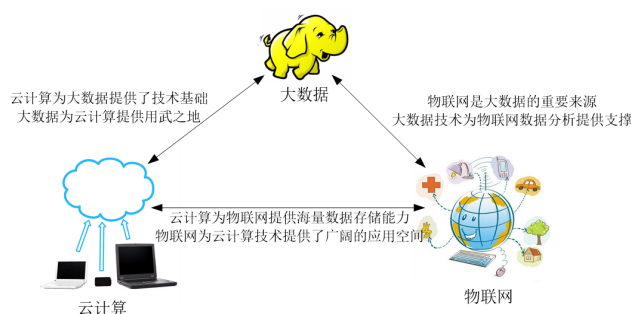
大数据计算模式	解决问题	代表产品
批处理计算	针对大规模数据的批量处理	MapReduce、Spark等
流计算	针对流数据的实时计算	Flink、Storm、S4、Flume、Streams、Puma、DStream、Super Mario、银河流数据处理平台等
图计算	针对大规模图结构数据的处理	Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb等
查询分析计算	大规模数据的存储管理和查询分析	Dremel、Hive、Cassandra、Impala等



云计算关键技术包括：虚拟化、分布式存储、分布式计算、多租户,数据中心是云计算的重要载体
物联网是物物相连的物联网，是互联网的延伸，它利用局部网络和互联网等。分为四层：感知层，网络层，处理层，应用层。
云计算、大数据和物联网代表了IT领域最新的技术发展趋势，三者既有区别又有联系

第一，大数据、云计算和物联网的区别。大数据侧重于对海量数据的存储、处理与分析，从海量数据中发现价值，服务于生产和生活；云计算本质上旨在整合和优化各种 IT 资源，并通过网络以服务的方式廉价地提供给用户；物联网的发展目标是实现物物相连，应用创新是物联网发展的核心。

第二，大数据、云计算和物联网的联系。从整体上看，大数据、云计算和物联网这三者是相辅相成的。大数据根植于云计算，大数据分析的很多技术都来自于云计算，云计算的分布式数据存储和管理系统（包括分布式文件系统和分布式数据库系统）提供了海量数据的存储和管理能力，分布式并行处理框架 MapReduce 提供了海量数据分析能力，没有这些云计算技术作为支撑，大数据分析就无从谈起。反之，大数据为云计算提供了“用武之地”，没有大数据这个“练兵场”，云计算技术再先进，也不能发挥它的应用价值。物联网的传感器源源不断产生的大量数据，构成了大数据的重要数据来源，没有物联网的飞速发展，就不会带来数据产生方式的变革，即由人工产生阶段转向自动产生阶段，大数据时代也不会这么快到来。同时，物联网需要借助于云计算和大数据技术，实现物联网大数据的存储、分析和处理。



第2章 大数据处理架构Hadoop

Hadoop是一个由Apache基金会所开发的分布式系统基础架构

特点：高可靠性，高效性，高可拓展性，高容错性，成本低，运行在Linux上，支持多种编程语言

主要解决：海量数据的存储和海量数据的分析计算问题

在Hadoop2.x时代，增加了分布式资源调度管理框架Yarn。Yarn只负责资源的调度，MapReduce只负责运算。

Hadoop基本安装配置：

- 1、创建Hadoop用户
- 2、安装java
- 3、设置SSH登录权限
- 4、单机安装配置
- 5、伪分布式安装配置

HDFS架构概述

1) NameNode (nn)：存储文件的元数据，如文件名，文件目录结构，文件属性（生成时间、副本数、文件权限），以及每个文件的块列表和块所在的DataNode等。

2) DataNode(dn)：在本地文件系统存储文件块数据，以及块数据的校验和。

3) Secondary NameNode(2nn)：用来监控HDFS状态的辅助后台程序，每隔一段时间获取HDFS元数据的快照。

NameNode：负责协调集群中的数据存储

DataNode：存储被拆分的数据块

JobTracker：协调数据计算任务

TaskTracker：负责执行由JobTracker指派的任务

SecondaryNameNode：帮助NameNode收集文件系统运行的状态信息

YARN架构

1) ResourceManager (RM) 主要作用：

1. 处理客户端请求
2. 监控NodeManager
3. 启动或监控ApplicationMaster
4. 资源的分配和调度

2) NodeManager (NM) 主要作用：

1. 管理单个节点上的资源
2. 处理来自ResourceManager的命令
3. 处理来自ApplicationMaster的命令

3) ApplicationMaster (AM) 作用：

1. 负责数据切分
2. 为应用程序申请资源并分配给内部的任务
3. 任务的监控与容错

4) Container

container是Yarn中的资源抽象，它封装了某个节点上的多维度资源，如内存，CPU，磁盘，网络等

此，需要在HDFS中为hadoop用户创建一个用户目录，命令如下：

```
Shell 命令
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -mkdir -p /user/hadoop
```

该命令中表示在HDFS中创建一个“/user/hadoop”目录，“-mkdir”是创建目录的操作，“-p”表示如果是多级目录，则父目录和子目录一起创建，这里“/user/hadoop”就是一个多级目录，因此必须使用参数“-p”，否则会出错。

“/user/hadoop”目录就成为hadoop用户对应的用户目录，可以使用如下命令显示HDFS中与当前用户hadoop对应的用户目录下的内容：

Hadoop包括三种安装模式：

单机模式：只在一台机器上运行，存储是采用本地文件系统，没有采用分布式文件系统HDFS；

伪分布式模式：存储采用分布式文件系统HDFS，但是，HDFS的名称节点和数据节点都在同一台机器上；

分布式模式：存储采用分布式文件系统HDFS，而且，HDFS的名称节点和数据节点位于不同机器上。

第3章 分布式文件系统HDFS

HDFS要实现以下目标：

兼容廉价的硬件设备，流数据读写，大数据集，简单的文件模型，强大的跨平台兼容性

局限性：不适合低延迟数据访问，无法高效存储大量小文件，不支持多用户写入及任意修改文件

HDFS默认一个块64MB，一个文件被分成多个块，以块作为存储单位——>最小化寻址开销

分布式文件系统的设计需求：透明性，并发性，文件复制（多副本），硬件和操作系统的异构性，可伸缩性，容错，安全

HDFS要实现的目标：1、兼容廉价的硬件设备，流数据读写，大数据集，简单的文件模式，强大的跨平台兼容性，不适合低延迟数据访问，无法高效存储大量小文件，不支持多用户写入及任意修改文件
组件：

NameNode	DataNode
• 存储元数据	• 存储文件内容
• 元数据保存在内存中	• 文件内容保存在磁盘
• 保存文件,block，datanode	• 维护了block id到datanode本地文件的映射关系

在HDFS中，名称节点（NameNode）负责管理分布式文件的命名空间（Namespace），保存了两个核心的数据结构，即FsImage和EditLog

FsImage用于维护文件系统树以及文件树中所有的文件和文件夹的元数据

操作日志文件EditLog中记录了所有针对文件的创建、删除、重命名等操作

数据节点是分布式文件系统HDFS的工作节点，负责数据的存储和读取。

名称节点运行期间EditLog不断变大的问题：解决？答案是：SecondaryNameNode第二名称节点
两个方面的功能:1、完成EditLog和FsImage的合并操作，减少EditLog文件大小，缩短名称节点重启时间；

2、作为名称节点的“检查点”，保存名称节点中的元数据信息。

第二名称节点是HDFS架构中的一个组成部分，它是用来保存名称节点中对HDFS元数据信息的备份，并减少名称节点重启的时间。SecondaryNameNode一般是单独运行在一台机器上

HDFS利用文件名找到对应的数据块，再根据每个数据块信息找到实际存储各个数据块的数据节点的位置，并把数据节点位置发送给客户端，最后客户端直接访问这些数据节点获取数据。名称节点并不参与数据的传输，这样设计能够在不同的数据节点上实现并发访问。

HDFS的命名空间包含目录、文件和块

数据节点：工作节点，负责数据的存储和读取，会根据客户端或者是名称节点的调度来进行数据的存储和检索，并且向名称节点定期发送自己所存储的块列表

HDFS采用了主从（Master/Slave）结构模型，一个HDFS集群包括一个名称节点（NameNode）和若干个数据节点（DataNode）。名称节点负责管理分布式文件的命名空间；数据节点是分布式文件系统HDFS的工作节点，负责数据的存储和读取。

HDFS 是一个部署在集群上的分布式文件系统，所有的HDFS通信协议都是构建在TCP/IP协议基础之上的

客户端与数据节点的交互是通过**RPC (Remote Procedure Call)** 来实现的。名称节点不会主动发起RPC，而是响应来自客户端和数据节点的RPC请求

唯一名称节点局限：（1）命名空间的限制（受内存大小影响）（2）性能的瓶颈（3）隔离问题（4）集群的可用性

HDFS存储原理：数据的冗余存储，数据存取策略，数据错误与恢复。

HDFS采用了**多副本方式**对数据进行冗余存储：（1）加快数据传输速度（2）容易检查数据错误（3）保证数据可靠性

为了提高数据的可靠性与系统的可用性，以及充分利用网络带宽，HDFS采用以**机架**为基础的数据存放策略

HDFS的数据复制采用了流水线复制的策略。

1. 名称节点出错：HDFS设置了备份机制，把这些核心文件同步复制到备份服务器

SecondaryNameNode上。当名称节点出错时，就可以根据备份服务器SecondaryNameNode中的FsImage和Editlog数据进行恢复。

2. 数据节点出错：名称节点会定期检查这种情况，**一旦发现某个数据块的副本数量小于冗余因子**，就会启动数据冗余复制，为它生成新的副本。（HDFS与其他分布式文件系统的最大区别就是可以调整冗余数据的位置）

3. 数据出错：当客户端读取文件的时候，会先读取该**信息文件**，然后，利用该信息文件对每个读取的数据块进行**校验**，如果校验出错，客户端就会请求到另外一个数据节点读取该文件块，并且向名称节点报告这个文件块有错误，名称节点会定期检查并且重新复制这个块（**校验码**）

在HDFS文件系统中DistributedFileSystem具体实现了FileSystem。

读取文件：

```
try {
    Configuration conf = new Configuration();
    conf.set("fs.defaultFS", "hdfs://localhost:9000");
    conf.set("fs.hdfs.impl", "org.apache.hadoop.hdfs.DistributedFileSystem");    FileSystem fs =
    FileSystem.get(conf);
    Path file = new Path("test");
    FSDataInputStream getIt = fs.open(file);
    BufferedReader d = new BufferedReader(new InputStreamReader(getIt));
    String content = d.readLine(); //读取文件一行        System.out.println(content);

    d.close(); //关闭文件
    fs.close(); //关闭hdfs
} catch (Exception e) {
    e.printStackTrace();
}
```

写入文件：


```

try {
Configuration conf = new Configuration();
conf.set("fs.defaultFS", "hdfs://localhost:9000");
conf.set("fs.hdfs.impl", "org.apache.hadoop.hdfs.DistributedFileSystem");    FileSystem fs =
FileSystem.get(conf);
byte[] buff = "Hello world".getBytes(); // 要写入的内容
String filename = "test"; //要写入的文件名                                FSDDataOutputStream os = fs.create(new
Path(filename));                os.write(buff,0,buff.length);
System.out.println("Create:"+ filename);
os.close();
fs.close();
} catch (Exception e) {
e.printStackTrace();
}

```

启动Hadoop: start -dfs.sh

关闭Hadoop: stop -dfs.sh

hadoop fs适用于任何不同的文件系统，比如本地文件系统和HDFS文件系统

hadoop dfs只能适用于HDFS文件系统

hdfs dfs只能适用于HDFS文件系统

hadoop fs -ls <path>:显示<path>指定的文件的详细信息

hadoop fs -mkdir <path>:创建<path>指定的文件夹

hadoop fs -cat <path>:将<path>指定的文件的内容输出到标准输出

hadoop fs -copyFromLocal <localsrc> <dst>:将本地源文件<localsrc>复制到路径<dst>指定的文件或文件夹中

hadoop fs -copyToLocal <target><localdst>: 将目标文件<target>复制到本地文件或者文件夹<localdst>中

hadoop fs -cp <src><dst>: 将文件从hdfs源路径<src>复制到hdfs目标路径<dst>

hadoop fs -get <src><localdst>: 将scr中的文件下载到本地目录<localdst>中

hadoop fs -put <localsrc><dst>: 从本地文件系统中将<localsrc>中的文件上传到HDFS的路径<dst>下（不删除本地，若将-put换成-moveFromLoca会删除本地文件）

hadoop fs -rm<path>: 删除path下的非空目录和文件

hadoop fs -rm -r <path>: 删除path下所有的目录

可以通过浏览器登录“<http://localhost:9870>”访问HDFS文件系统

eg:


```
hadoop fs -ls /          #查看HDFS的根目录
hadoop fs -ls -R /        #递归查看HDFS的根目录/
hadoop fs -mkdir /data    #在HDFS中创建一个data目录
hadoop fs -touchz /data/a.txt #在HDFS的/data目录下创建一个空文件a.txt
hadoop fs -cp /data/a.txt /data/b.txt #将/data目录下的a.txt复制衣服跟在当前目录下重命名为b.txt
hadoop fs -mv /data/a.txt /      #将HDFS的/data目录下的a.txt移动到HDFS的根目录
hadoop fs -cat /a.txt      #查看HDFS根目录下的a.txt，由于刚刚创建的是空内容，显示没有内容
hadoop fs -get /a.txt ~/    #将HDFS根目录下的a.txt下载到本地/home目录（本地指的是虚拟机环境）
hadoop fs -put ~/a.txt /data/ #将HOME目录下的a.txt上传到HDFS中的data目录下
```

第4章 分布式数据库HBase

为什么还要去设计HBase这么一个数据库产品呢

原因

- 虽然已经有了HDFS和MapReduce，但是 Hadoop主要解决大规模数据离线批量处理Hadoop是没有办法满足大数据实时处理需求的随着这些年数据的大规模爆炸式增长，传统关系型数据库的扩展能力非常有限

Region到底被存到哪里去了？

HBase设计了三层结构实现Region的寻址和定位

- 首先要构建一个元数据表，假设这个元数据表只有两列
第一列是Region的id第二列是Region服务器的id
- HBase最开始构建时有一个映射表，这个映射表被称为.META.表
- .META.表是用于存储元数据的

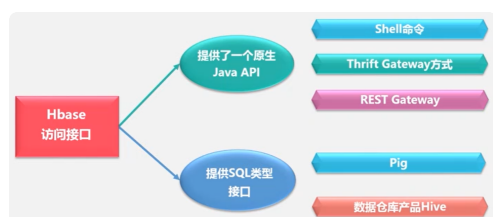
HDFS面向批量询问模式，不是随机访问模式，只有HBase能满足不断增长的数据需求

BigTable是一个分布式存储系统，起初用于解决典型的互联网搜索问题。

HBase主要用来存储非结构化和半结构化的松散数据。目标是处理非常庞大的表，可以利用廉价计算机集群处理由超过10亿行数据和数百万列元素组成的数据表。

表4-1 HBase和BigTable的底层技术对应关系

	BigTable	HBase
文件存储系统	GFS	HDFS
海量数据处理	MapReduce	Hadoop MapReduce
协同服务管理	Chubby	Zookeeper



• HBase与传统关系数据库的区别：

(1) **数据类型**：关系数据库采用关系模型，具有丰富的数据类型和存储方式；HBase则采用了更加简单的数据模型，它把数据存储为未经解释的字符串。

(2) **数据操作**：关系数据库中包含了丰富的操作，其中会涉及复杂的多表连接；HBase操作则不存在复杂的表与表之间的关系，只有简单的插入、查询、删除、清空等。

(3) **存储模式**：关系数据库是基于行模式存储的；HBase是基于列存储的，每个列族都由几个文件保存，不同列族的文件是分离的。

(4) **数据索引**：关系数据库通常可以针对不同列构建复杂的多个索引，以提高数据访问性能；HBase只有一个索引——行键。

(5) **数据维护**：关系数据库中更新操作会用最新的当前值去替换记录中原来的旧值。而在HBase中执行更新操作时，并不会删除数据旧的版本，而是生成一个新的版本。

(6) **可伸缩性**：HBase和BigTable这些分布式数据库就是为了实现灵活的水平扩展而开发的，能够轻易地通过在集群中增加或者减少硬件数量来实现性能的伸缩。

4.2 HBase访问接口

类型	特点	场合
Native Java API	最常规和高效的访问方式	适合Hadoop MapReduce作业并行批处理HBase表数据
HBase Shell	HBase的命令行工具，最简单的接口	适合HBase管理使用
Thrift Gateway	利用Thrift序列化技术，支持C++、PHP、Python等多种语言	适合其他异构系统在线访问HBase表数据
REST Gateway	解除了语言限制	支持REST风格的Http API访问HBase
Pig	使用Pig Latin流式编程语言来处理HBase中的数据	适合做数据统计
Hive	简单	当需要以类似SQL语言方式来访问HBase的时候

• HBase数据模型

HBase是一个稀疏、多维度、排序的映射表，这张表的索引是行键、列族、列限定符和时间戳。每个值是一个未经解释的字符串，没有数据类型。每一行都有一个可排序的行键和任意多的列。表在水平方向由一个或多个列族组成，一个列族中可以包含任意多个列，同一个列族里面的数据存储在一起来。列族支持动态扩展，可以添加一个列族或列，无需预先定义列的数量以及类型，所有列均以字符串形式存储。

- 数据模型相关概念：

表：HBase采用表来组织数据，表由行和列组成，列划分为若干个列族。

行：每个行由行键（row key）来标识。

列族：是基本的访问控制单元。

列限定符：列族里的数据通过列限定符来定位。

单元格：在HBase表中，通过行、列族和列限定符确定一个“单元格”，单元格中的数据无数据类型，被视为字节数组byte[]。

时间戳：每个单元格都保存着同一份数据的多个版本，这些版本采用时间戳进行索引。

- 数据坐标：

HBase中需要根据行键、列族、列限定符和时间戳来确定一个单元格，可以视为一个“**四维坐标**”，即[行键, 列族, 列限定符, 时间戳]。

- HBase的实现包括三个主要的功能组件：

- (1) 库函数：链接到每个客户端；
- (2) 一个Master主服务器；
- (3) 许多个Region服务器。

主服务器Master负责管理和维护HBase表的分区信息，维护Region服务器列表，分配Region，负载均衡。

Region服务器负责存储和维护分配给自己的Region，处理来自客户端的读写请求。

客户端并不是直接从Master主服务器上读取数据，而是在获得Region的存储位置信息后，直接从Region服务器上读取数据。

客户端并不依赖Master，而是通过Zookeeper来获得Region位置信息，大多数客户端甚至从来不和Master通信，这种设计方式使得Master负载很小。

一个HBase表被划分成多个Region

4.4.2 表和Region

每个Region默认大小是100MB到200MB（2006年以前的硬件配置）。目前每个Region最佳大小建议**1GB-2GB**

每个Region的最佳大小**取决于单台服务器的有效处理能力。**

同一个Region不会被分拆到多个Region服务器。每个Region服务器存储10-1000个Region。

4.4.3 Region的定位

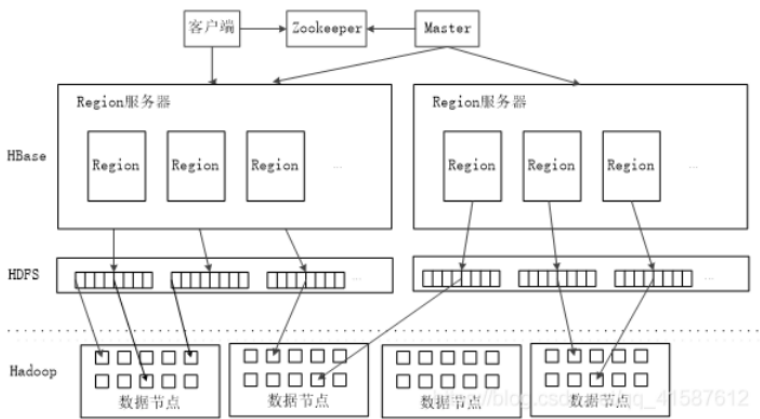
元数据表，又名.META.表，存储了Region和Region服务器的映射关系。当HBase表很大时，.META.表也会被分裂成多个Region。

根数据表，又名-ROOT-表，记录所有元数据的具体位置。-ROOT-表只有唯一 一个Region，名字是在程序中被写死的。Zookeeper文件记录了-ROOT-表的位置。

HBase的三层结构中各层次名称和作用		
层次	名称	作用
第一层	Zookeeper文件	记录了-ROOT-表的位置信息
第二层	-ROOT-表	记录了.META.表的Region位置信息 -ROOT-表只能有一个Region。通过-ROOT-表， 就可以访问.META.表中的数据
第三层	.META.表	记录了用户数据表的Region位置信息， .META.表可以有多个Region，保存了HBase中所有用户数 据表的Region位置信息

为了加速寻址，客户端会缓存位置信息，同时，需要解决缓存失效问题。
寻址过程客户端只需要询问Zookeeper服务器，不需要连接Master服务器。
客户端访问数据时的“三级寻址”

4.5 HBase运行机制



- 试述HBase系统基本架构以及每个组成部分的作用？
- 1).客户端 客户端包含访问HBase的接口，同时在缓存中维护着已经访问过的Region位置信息，用来加快后续数据访问过程。
 - 2).Zookeeper服务器 Zookeeper可以帮助选举出一个Master作为集群的总管，并保证在任何时刻总有唯一一个Master在运行，这就避免了Master的“单点失效”问题。
 - 3).Master 负责管理和维护HBase表的分区信息，维护Region服务器列表，分配Region，负载均衡。
 - 4).Region服务器 负责存储和维护分配给自己的Region，处理来自客户端的读写请求。

- 阐述Region服务器向HDFS文件系统读写数据的基本原理？

1、用户读写数据过程：①用户写入数据时，被分配到相应Region服务器去执行②用户数据首先被写入到MemStore和Hlog中③只有当操作写入Hlog之后，commit()调用才会将其返回给客户端④当用户读取数据时，Region服务器会首先访问MemStore缓存，如果找不到，再去磁盘上面的StoreFile中寻找

2、缓存的刷新：①系统会周期性地把MemStore缓存里的内容刷写到磁盘的StoreFile文件中，清空缓存，并在Hlog里面写入一个标记

②每次刷写都生成一个新的StoreFile文件，因此，每个Store包含多个StoreFile文件

③每个Region服务器都有一个自己的HLog文件，每次启动都检查该文件，确认最近一次执行缓存刷新操作之后是否发生新的写入操作；如果发现更新，则先写入MemStore，再刷写到StoreFile，最后删除旧的Hlog文件，开始为用户提供服务

3、StoreFile的合并：①每次刷写都生成一个新的StoreFile，数量太多，影响查找速度

②调用Store.compact()把多个合并成一个

③合并操作比较耗费资源，只有数量达到一个阈值才启动合并

- Store工作原理

Store是Region服务器的核心。

1) 多个StoreFile合并成一个。

2) 单个StoreFile过大时，又触发分裂操作，1个父Region被分裂成两个子Region。

- HLog工作原理

分布式环境必须要考虑系统出错。HBase采用HLog保证系统恢复。

HBase系统为**每个Region服务器**配置了一个HLog文件，它是一种预写式日志。用户更新数据必须首先写入日志后，才能写入MemStore缓存。

Zookeeper会实时监测每个Region服务器的状态，当某个Region服务器发生故障时，Zookeeper会通知Master。Master首先会处理该故障Region服务器上面**遗留的HLog文件**，这个遗留的HLog文件中包含了来自多个Region对象的日志记录。

共用日志优点：提高对表的写操作性能；缺点：**恢复时需要分拆日志**。



启动hbase shell

```

hadoop@ubuntu:~$ hbase shell
# 创建表t: 列族为f, 列族版本号为5
hbase> create 't1',{NAME => 'f1',VERSIONS => 5}
# 创建表t: 列族为f1、f2、f3, 两种方式等价
hbase> create 't1', {NAME => 'f1'}, {NAME => 'f2'}, {NAME => 'f3'}
hbase> create 't1', 'f1', 'f2', 'f3'
# 创建表t: 将表根据分割算法HexStringSplit 分布在15个Region里
hbase> create 't1', 'f1', {NUMREGIONS => 15, SPLITALGO => 'HexStringSplit'}
# 创建表t: 指定Region的切分点
hbase> create 't1', 'f1', SPLITS => ['10', '20', '30', '40']
# list 列出所有表
hbase> list
# put 向表中指定的单元格添加数据
hbase> put 't1','row1','f1:c1',120000      # 通过表, 行键, 列族: 列限定符进行定位, 值为120000
# get 通过指定坐标来获取单元格的值
hbase(main):005:0> get 't1','row1','f1:c1'
COLUMN          CELL
f1:c1           timestamp=1609810077099, value=120000
1 row(s)
Took 0.0722 seconds
# delete 删除表中指定单元格的数据
hbase(main):021:0> delete 't1','row1','f1:c1',timestamp=1609810077099
# scan 浏览表的信息
hbase(main):004:0> scan 't1'              # 这时会显示表t1中的所有行
# scan 浏览某个单元格的数据
hbase(main):010:0> scan 't1',{COLUMNS => 'f1:c1'}
# alter 修改列族模式
hbase(main):011:0> alter 't1',NAME => 'f2'      # 向表t1中增加列族f2
hbase(main):014:0> alter 't1',NAME => 'f2',METHOD => 'delete' # 将表t1中的列族f2删除
# count 统计表中的行数
hbase(main):015:0> count 't1'              # 统计t1的行数
# describe 显示表的相关信息
hbase(main):017:0> describe 't1'
# enable/disable 使表有效或无效
hbase(main):015:0> disable 't1'
# drop 删除表,这里需要注意删除表之前要先使用disable使这个表无效, 这也是为了防止误删

```

```
hbase(main):023:0> disable 't1'
Took 0.8378 seconds
hbase(main):024:0> drop 't1'
Took 0.4997 seconds
# exists 判断某个表是否存在
hbase(main):025:0> exists 't1'
Table t1 does not exist
Took 0.0231 seconds
=> false
# truncate 使表无效并清空该表的数据
hbase(main):029:0> truncate 'teacher'
hbase(main):031:0> exists 'teacher'    # truncate后查看该表是否存在
Table teacher does exist
Took 0.0156 seconds
=> true                                # 还存在
# 查看HBase集群状态
hbase(main):026:0> status
1 active master, 0 backup masters, 1 servers, 0 dead, 5.0000 average load
Took 0.0582 seconds
# 退出hbase shell
hbase> exit
```

第5章 NoSQL数据库

ACID：原子性，一致性，隔离性，持续性

- NoSQL简介

NoSQL数据库具有以下几个特点：（1）灵活的可扩展性（2）灵活的数据模型（3）与云计算紧密融合
现在已经有公司使用了NoSQL数据库：Google、Facebook、Mozilla、Adobe、Foursquare、LinkedIn、Digg、McGraw-Hill Education、Vermont Public Radio、百度、腾讯、阿里、新浪、华为.....

- NoSQL兴起的原因

1、关系数据库已经无法满足Web2.0的需求。主要表现在以下几个方面：

- （1）无法满足海量数据的管理需求
- （2）无法满足数据高并发的需求
- （3）无法满足高可扩展性和高可用性的需求

2、关系数据库的关键特性包括完善的事务机制和高效的查询机制，到了Web2.0时代却成了鸡肋：

- (1) Web2.0网站系统通常不要求严格的数据库事务
- (2) Web2.0并不要求严格的读写实时性
- (3) Web2.0通常不包含大量复杂的SQL查询（去结构化，存储空间换取更好的查询性能）

• NoSQL与关系数据库的比较

比较标准	RDBMS	NoSQL	备注
数据库原理	完全支持	部分支持	RDBMS有关系代数理论作为基础。NoSQL没有统一的理论基础。
数据规模	大	超大	RDBMS很难实现横向扩展，纵向扩展的空间也比较有限，性能会随着数据规模的增大而降低。NoSQL可以很容易通过添加更多设备来支持更大规模的数据。
数据库模式	固定	灵活	RDBMS需要定义数据库模式，严格遵守数据定义和相关约束条件。NoSQL不存在数据库模式，可以自由灵活定义并存储各种不同类型的数据。
查询效率	快	可以实现高效的简单查询，但是不具备高度结构化查询等特性，复杂查询的性能不尽人意。	RDBMS借助索引机制可以实现快速查询（包括记录查询和范围查询）。很多NoSQL数据库没有面向复杂查询的索引，虽然NoSQL可以使用MapReduce来加速查询，但是，在复杂查询方面的性能仍然不如RDBMS。
一致性	强一致性	弱一致性	RDBMS严格遵守事务ACID模型，可以保证事务强一致性。很多NoSQL数据库放松了对事务ACID四性的要求，而是遵守BASE模型，只能保证最终一致性。
数据完整性	容易实现	很难实现	任何一个RDBMS都可以很容易实现数据完整性，比如通过主键或者非空约束来实现实体完整性，通过主键、外键来实现参照完整性，通过约束或者触发器来实现用户自定义完整性。但是，在NoSQL数据库却无法实现。
扩展性	一般	好	RDBMS很难实现横向扩展，纵向扩展的空间也比较有限。NoSQL在设计之初就充分考虑了横向扩展的需求，可以很容易通过添加廉价设备实现扩展。
可用性	好	很好	RDBMS在任何时候都以保证数据一致性为优先目标，其次才是优化系统性能，随着数据规模的增大，RDBMS为了保证严格的一致性，只能提供相对较弱的可用性。大多数NoSQL都能提供较高的可用性。
标准化	是	否	RDBMS已经标准化（SQL）。NoSQL还没有行业标准，不同的NoSQL数据库都有自己的查询语言，很难规范应用程序接口。StoneBraker认为：NoSQL缺乏统一查询语言，将会拖累NoSQL发展。
技术支持	高	低	RDBMS经过几十年的发展，已经非常成熟，Oracle等大型厂商都可以提供很好的技术支持。NoSQL在技术支持方面仍然处于起步阶段，还不成熟，缺乏有力的技术支持。
可维护性	复杂	复杂	RDBMS需要专门的数据库管理员(DBA)维护。NoSQL数据库虽然没有DBMS复杂，也难以维护。

• 总结：

(1) 关系数据库

优势：以完善的关系代数理论作为基础，有严格的标准，支持事务ACID四性，借助索引机制可以实现高效的查询，技术成熟，有专业公司的技术支持。

劣势：可扩展性较差，无法较好支持海量数据存储，数据模型过于死板、无法较好支持Web2.0应用，事务机制影响了系统的整体性能等。

关系数据库应用场景：电信、银行等领域的关键业务系统，需要保证强事务一致性。

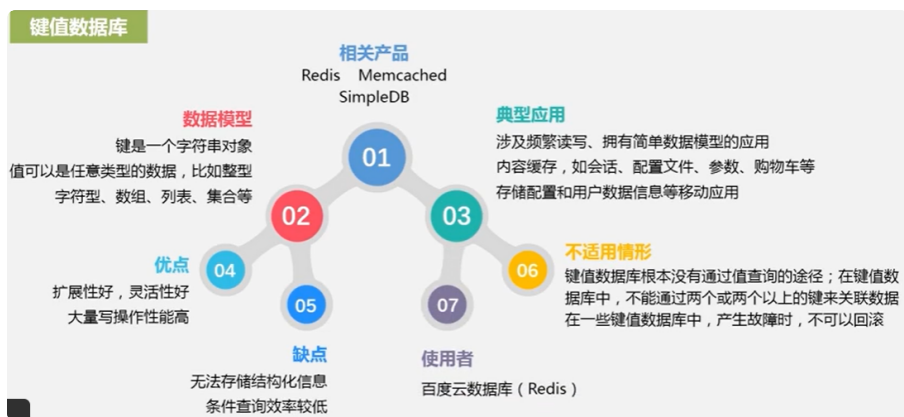
(2) NoSQL数据库

优势：可以支持超大规模数据存储，灵活的数据模型可以很好地支持Web2.0应用，具有强大的横向扩展能力等。

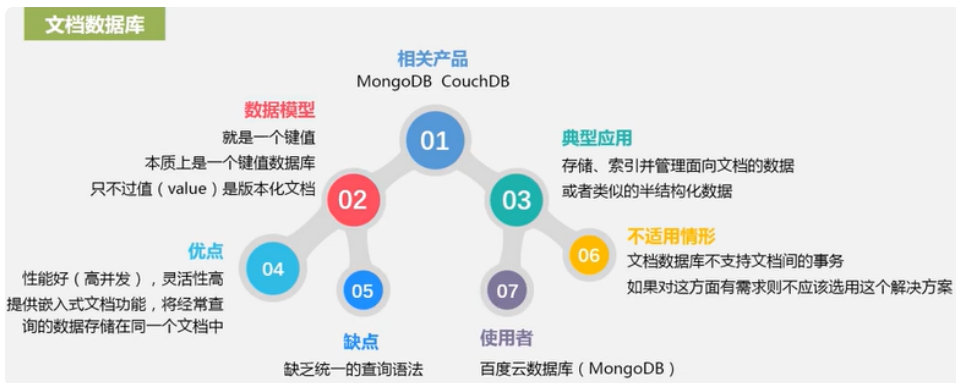
劣势：缺乏数学理论基础，复杂查询性能不高，大都不能实现事务强一致性，很难实现数据完整性，技术尚不成熟，缺乏专业团队的技术支持，维护较困难等。

NoSQL数据库应用场景：互联网企业、传统企业的非关键业务（比如数据分析）

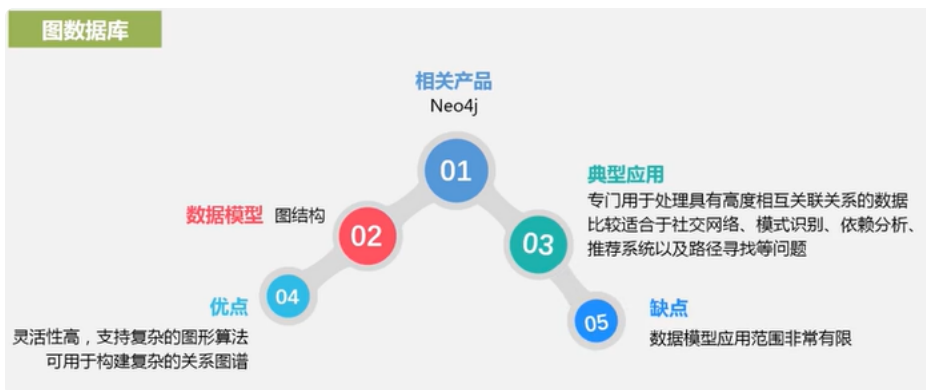
- 典型的NoSQL数据库通常包括键值数据库、列族数据库、文档数据库和图形数据库。



键值数据库成为理想的缓冲层解决方案



文档数据库最像关系数据库，特性：能够将自己的数据的内容和类型进行自我描述。文档数据库可以完整包含在一个文档里，具有较好的并发性。在对数据进行更新时，只需要锁定一个文档就可以把相关数据修改掉。



MySQL	MongoDB	HBase	Redis
功能较稳定强大 满足多样需求	数据模型较灵活 支持较多功能	具有很好的扩展性 依赖Hadoop生态环境	模型相对较为简单，可提供随机 数据存储，数据库伸缩性较好

NoSQL的三大理论基石：CAP,BASE,最终一致性

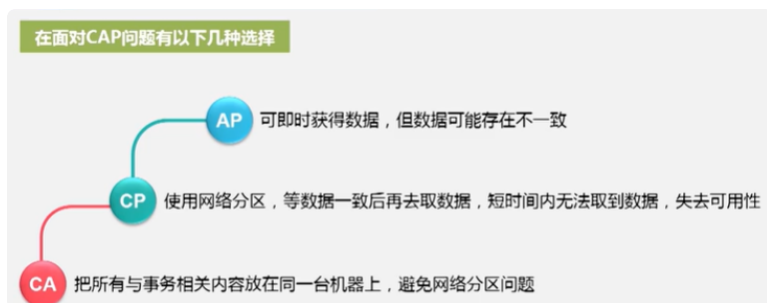
- CAP（必考！！！！）

C（Consistency一致性）：是指任何一个读操作总是能够读到之前完成的写操作的结果，也就是在分布式环境中，多点的数据是一致的，或者说，所有节点在同一时间具有相同的数据；

A（Availability可用性）：是指快速获取数据，可以在确定的时间内返回操作结果，保证每个请求不管成功或者失败都有响应；

P（Tolerance of Network Partition分区容忍性）：是指当出现网络分区的情况时（即系统中的一部分节点无法和其他节点进行通信），分离的系统也能够正常运行，也就是说，系统中任意信息的丢失或失败不会影响系统的继续运作。

CAP理论告诉我们，一个分布式系统不可能同时满足一致性、可用性和分区容忍性这三个需求，最多只能同时满足其中两个。



1、MySQL, SQL Server PostgreSQL

2、Neo4j、BigTable, HBase等NoSQL

3、Dynamo, Riak, CouchDB, Cassandra

- 一个数据库事务具有ACID四性：

A（Atomicity）：原子性，是指事务必须是原子工作单元，对于其数据修改，要么全都执行，要么全都不执行

C（Consistency）：一致性，是指事务在完成时，必须使所有的数据都保持一致状态

I（Isolation）：隔离性，是指由并发事务所做的修改必须与任何其它并发事务所做的修改隔离

D（Durability）：持久性，是指事务完成之后，它对于系统的影响是永久性的，该修改即使出现致命的系统故障也将一直

- BASE的基本含义是：

基本可用（Basically Available）：是指一个分布式系统的一部分发生问题变得不可用时，其他部分仍然可以正常使用，也就是允许分区失败的情形出现。

软状态（Soft-state）：是与硬状态（Hard-state）相对应的一种提法。数据库保存的数据是“硬状态”时，可以保证数据一致性，即保证数据一直是正确的。“软状态”是指状态可以有一段时间不同步，具有一定的滞后性。

最终一致性（Eventual consistency）：一致性的类型包括强一致性和弱一致性，二者的主要区别在于高并发的数据访问操作下，后续操作是否能够获取最新的数据。对于强一致性而言，当执行完一次更新操作后，后续的其他读操作就可以保证读到更新后的最新数据；反之，如果不能保证后续访问读到的都

是更新后的最新数据，那么就是弱一致性。而最终一致性只不过是弱一致性的一种特例，允许后续的访问操作可以暂时读不到更新后的数据，但是经过一段时间之后，必须最终读到更新后的数据。

- 最终一致性

最终一致性根据更新数据后各进程访问到数据的时间和方式的不同，又可以区分为以下5类：

因果一致性：如果进程A通知进程B它已更新了一个数据项，那么进程B的后续访问将获得A写入的最新值。而与进程A无因果关系的进程C的访问，仍然遵守一般的最终一致性规则。

“读己之所写”一致性：可以视为因果一致性的一个特例。当进程A自己执行一个更新操作之后，它自己总是可以访问到更新过的值，绝不会看到旧值。

单调读一致性：如果进程已经看到过数据对象的某个值，那么任何后续访问都不会返回在那个值之前的值。

会话一致性：它把访问存储系统的进程放到会话（session）的上下文中，只要会话还存在，系统就保证“读己之所写”一致性。如果由于某些失败情形令会话终止，就要建立新的会话，而且系统保证不会延续到新的会话。

单调写一致性：系统保证来自同一个进程的写操作顺序执行。系统必须保证这种程度的一致性，否则就非常难以编程了。

New数据库不仅具有NoSQL对海量数据的存储管理能力，还保持了传统数据库支持ACID和SQL等特性

第6章 云数据库

云计算是通过网络以服务的方式为用户提供非常廉价的IT资源

云计算八大优势：按需服务，随时服务，通用性，高可靠性，及其廉价，超大规模，虚拟化，高拓展性

1.云数据库的概念

云数据库是部署和虚拟化在云计算环境中的数据库。

云数据库是在云计算的大背景下发展起来的一种新兴的共享基础架构的方法，它极大地增强了数据库的存储能力，消除了人员、硬件、软件的重复配置，让软、硬件升级变得更加容易。云数据库具有高可扩展性、高可用性、采用多租形式和支持资源有效分发等特点。它允许用户以服务的方式通过网络获得云端的数据库功能。

2.云数据库的特性

- (1) 动态可扩展：用户按需扩展
- (2) 高可用性：云数据库具有故障自动单点切换、数据库自动备份等功能
- (3) 较低的使用代价：RDS支付的费用远低于自建数据库所需的成本
- (4) 易用性：提供WEB界面进行配置、操作数据库实例
- (5) 高性能
- (6) 免维护：有专门的维护人员
- (7) 安全

云数据库是个性化数据存储需求的理想选择。首先，云数据库可以满足大企业海量数据存储需求。其次，云数据库可以满足中小企业的低成本数据存储需求。另外，云数据库可以满足企业动态变化的数据存储需求。

云数据库与其他数据库的关系：从数据模型的角度来说，云数据库并非一种全新的数据库技术，而只是以服务的方式提供数据库功能。云数据库并没有专属于自己的数据模型，同一个公司也可能提供采用不同数据模型的多种云数据库服务。

云数据库产品

- 企业产品
- Amazon Dynamo、SimpleDB、RDS
- Google Google Cloud SQL
- Microsoft Microsoft SQL Azure
- 阿里 阿里云RDS
- 百度 百度云数据库
- 腾讯 腾讯云数据库

UMP系统是低成本和高性能的MySQL云数据库方案。

第7章 MapReduce

“摩尔定律”：CPU性能大约每隔18个月翻一番。

MapReduce是一种分布式并行编程模型，常用于处理大规模数据集。其将运行于分布式集群上的复杂的并行计算过程高度抽象为两个函数Map与Reduce，这两个函数思想都源自函数式编程思想。

	传统并行计算框架	MapReduce
集群架构/容错性	共享式(共享内存/共享存储)，容错性差	非共享式，容错性好
硬件/价格/扩展性	刀片服务器、高速网、SAN，价格贵，扩展性差	普通PC机，便宜，扩展性好
编程/学习难度	what-how，难	what，简单
适用场景	实时、细粒度计算、计算密集型	批处理、非实时、数据密集型

2.MapReduce的2个特点：分而治之、计算向数据靠拢（在做应用程序计算的时候，构建一个集群，在集群中选择相关的机器作为map机器，这些map机器负责相关的数据处理分析，还有一些是Reduce机器，做Reduce任务处理。）

而数据向计算靠拢：完成一次数据分析时，选择一个计算节点，把运行数据分析的程序放到计算节点上运行

3、MapReduce的架构：采用了master/slave的架构

Master/slave架构中包括：一个master服务器以及若干个slave服务器（Master就是名称节点，Slave就是数据节点）Master上面会运行作业跟踪器（JobTracker）；JobTracker负责整个作业的调度和处理

以及失败和恢复；slave上面会运行负责具体任务执行的组件（TaskTracker）；TaskTracker负责接收JobTracker给它发的作业处理指令，完成具体的任务处理

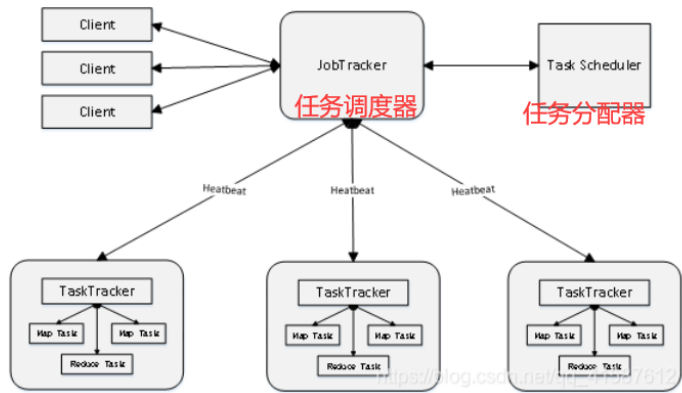
4、MapReduce将并行计算过程抽象到两个函数：Map和Reduce。

在MapReduce中，一个存储在分布式文件系统的大规模数据集，会被切分成许多独立的小数据块，小数据块可以被多个Map任务并行处理。MapReduce应用程序不一定要用Java来写。

注意：适合用MapReduce来处理的数据集需要满足一个前提条件：待处理的数据集可以分解成许多小的数据集，而且每一个小数据集都可以完全并行地进行处理。

函数	输入	输出
Map	<k1,v1> 如： <行号,"a b c">	List(<k2,v2>) 如： <"a",1> <"b",1> <"c",1>
Reduce	<k2,List(v2)> 如：<"a", <1,1,1>>	<k3,v3> <"a",3>

5、MapReduce体系结构：Client, JobTracker, TaskTracker, Task



1) Client

用户编写的MapReduce程序通过Client提交到JobTracker端。用户可通过Client提供的一些接口查看作业运行状态。

2) JobTracker

JobTracker负责资源监控和作业调度。

JobTracker 监控所有TaskTracker与Job的健康状况，一旦发现失败，就将相应的任务转移到其他节点。

JobTracker 会跟踪任务的执行进度、资源使用量等信息，并将这些信息告诉任务调度器（TaskScheduler），而调度器会在资源出现空闲时，选择合适的任务去使用这些资源。

3) TaskTracker

TaskTracker 会周期性地通过“心跳”将本节点上资源的使用情况和任务的运行进度汇报给JobTracker，同时接收JobTracker 发送过来的命令并执行相应的操作（如启动新任务、杀死任务等）。

TaskTracker使用一个槽的概念，叫slot。它会把机器上面所有的CPU内存资源进行打包，然后把资源进行等分，等分成很多slot。slot是一个资源调度单位。一个Task 获取到一个slot 后才有机会运行，而Hadoop调度器的作用就是将各个TaskTracker上的空闲slot分配给Task使用。slot 分为Map slot 和Reduce slot 两种。两种类型slot是不通用的，该缺陷Hadoop2.0中做出了修改。

4) Task

Task 分为Map Task 和Reduce Task 两种，均由TaskTracker 启动。

5) Task Schedule负责任务调度（解决应该把哪个Task（任务）分发给哪个TaskTracker去执行（各个TaskTracker是分布在不同的机器上面，负责执行具体任务）

Task Schedule是一个可插拔模块，允许用户自己去编写调度模块，采用自己的任务调度策略去实现

6、MapReduce工作流程

- 试述MapReduce的工作流程？

- 1).InputFormat模块做预处理，如验证输入格式是否符合输入定义，然后将输入文件切分成逻辑上的多个InputSplit。
- 2).通过RecordReader处理InputSplit中的具体记录，转换成键值对输入给Map任务。
- 3).Map任务根据用户定义的映射规则，输出一系列的 $\langle key, value \rangle$ 作为中间结果。
- 4).Shuffle过程将Map的输出进行一定的分区、排序、合并、归并等操作，将无序的 $\langle key, value \rangle$ 转到有序的 $\langle key, value - list \rangle$ 。
- 5).Reduce以一系列的 $\langle key, value - list \rangle$ 中间结果作为输入，执行用户定义的逻辑，输出结果给OutputFormat模块。
- 6).OutputFormat模块验证输出目录是否存在以及输出结果类型是否符合配置文件中的配置类型，都满足的话，输出Reduce结果到分布式文件系统。

注意：

- 1) 不同的Map任务之间**不会进行通信**。
- 2) 不同的Reduce任务之间也不会发生任何信息交换。
- 3) 用户**不能显式地**从一台机器向另一台机器发送消息。
- 4) 所有的数据交换都是通过MapReduce框架自身去实现的。

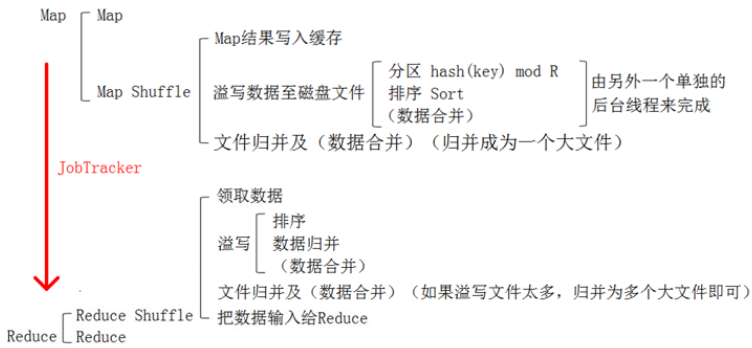
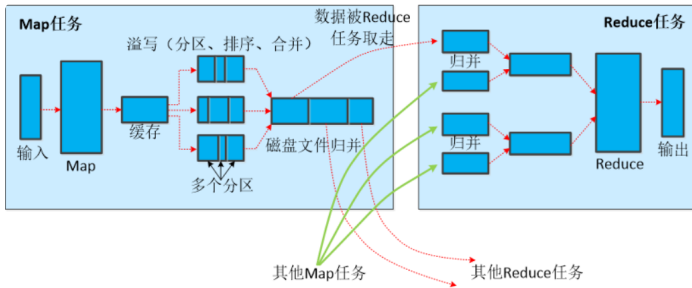
HDFS 以固定大小的block 为基本单位存储数据，而对于MapReduce 而言，其处理单位是split。split 是一个逻辑概念，它只包含一些元数据信息，比如数据起始位置、数据长度、数据所在节点等。它的划分方法完全由用户自己决定。（名称节点）**整个集群里面，分了多少个分片，就会产生多少个map任务。**

在MapReduce的整个执行过程中，Map任务的输入文件，Reduce任务的**处理结果**都是保存在分布式文件系统**中的**，而Map任务处理得到的**中间结果**则保存在**本地存储中**（如磁盘）

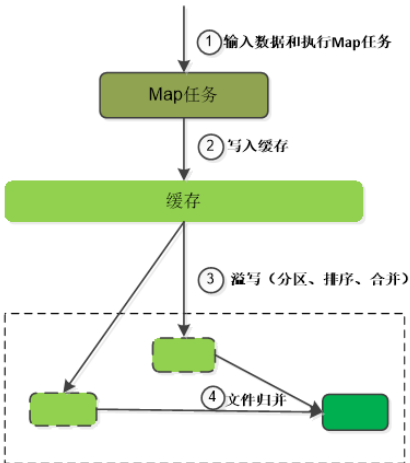
reduce任务如何设置？

最优的reduce任务个数**取决于集群中可用的reduce任务槽（slot）的数目。**

7、shuffle过程简介



2. Map端的Shuffle过程



- 每个Map任务分配一个缓存
- MapReduce默认100MB缓存

- 设置溢出比例0.8
- 分区默认采用哈希函数
- 排序是默认的操作
- 排序后可以合并(Combine)
- 合并不能改变最终结果

- 在Map任务全部结束之前进行归并
- 归并得到一个大的文件, 放在本地磁盘
- 文件归并时, 如果溢出文件数量大于预定值(默认是3)则可以再次启动Combiner, 少于3不需要
- JobTracker会一直监测Map任务的执行, 并通知Reduce任务来领取数据

合并(Combine)和归并(Merge)的区别:

两个键值对<"a",1>和<"a",1>, 如果合并, 会得到<"a",2>, 如果归并, 会得到<"a",<1,1>>

- 试分析Shuffle过程的作用？

Shuffle过程将Map的输出进行一定的分区、排序、合并、归并等操作并交给Reduce。

1).Map端的Shuffle过程 Map输出结果首先被写入缓存，缓存满时启动溢写操作，将缓存的数据进行分区，然后将每个分区进行排序和合并，之后写入磁盘文件。Map任务全部结束之前，这些文件会归并成一个大的磁盘文件，然后通知相应的Reduce任务来领取属于自己处理的数据。

2).Reduce端的Shuffle过程 Reduce任务从Map端的不同Map机器领会属于自己处理的数据，对数据进行归并后交给Reduce处理。

为什么要分区？ Map输出一些键值对，最终是扔给多个不同的reduce任务去处理，所以一定要分区。如果有4个reduce任务，就会分成四个分区，每个分区都由对应的reduce任务把它取走。这大的文件里面的键值对都是分区的而且是排序的。

合并是为了减少溢写到磁盘当中的数据量。由两个键值对合并完之后变为一个键值对，由此，可以减少很多键值对，这样写入的数据量就会大大减少

JobTracker会跟踪Map任务，JobTracker一旦检测到Map任务写完数据，JobTracker就会通知相应的Reduce任务，把这些属于该Reduce任务处理的分区数据让Reduce拉走。

3. Reduce端的Shuffle过程

- Reduce任务通过RPC向JobTracker询问Map任务是否已经完成，若完成，则领取数据
- Reduce领取数据先放入缓存，来自不同Map机器，先归并，再合并，写入磁盘
- 多个溢写文件归并成一个或多个大文件，文件中的键值对是排序的
- 当数据很少时，不需要溢写到磁盘，直接在缓存中归并，然后输出给Reduce

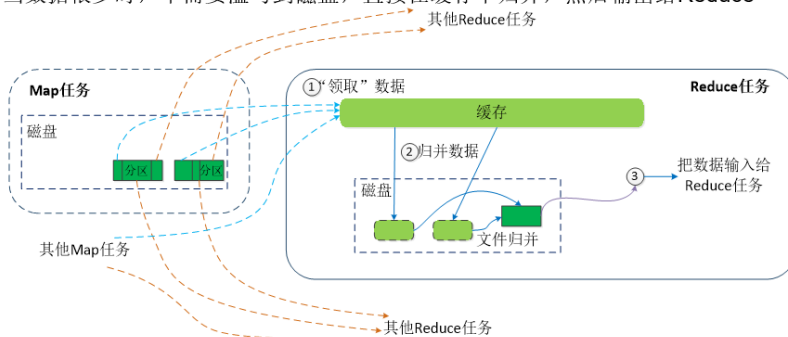


图7-5 Reduce端的Shuffle过程

8、实例：WordCount

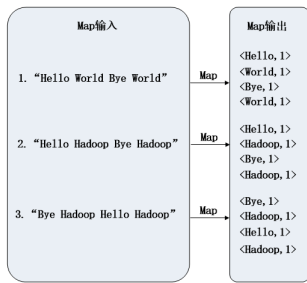


图7-7 Map过程示意图

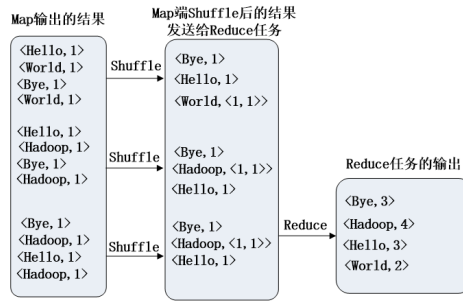


图7-8 用户没有定义Combiner时的Reduce过程示意图

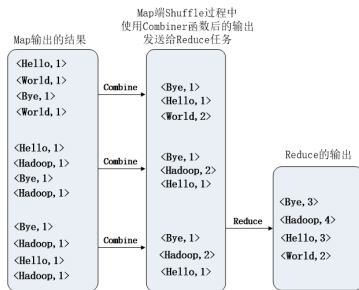


图7-9 用户有定义Combiner时的Reduce过程示意图

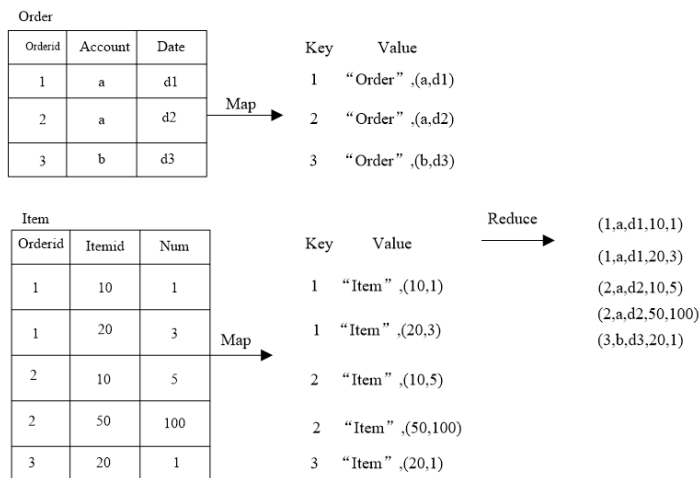
9、MapReduce的具体应用：自然连接

用MapReduce实现关系的自然连接

雇员			部门		雇员 ⋈ 部门			
Name	EmpId	DeptName	DeptName	Manager	Name	EmpId	DeptName	Manager
Harry	3415	财务	财务	George	Harry	3415	财务	George
Sally	2241	销售	销售	Harriet	Sally	2241	销售	Harriet
George	3401	财务	生产	Charles	George	3401	财务	George
Harriet	2202	销售			Harriet	2202	销售	Harriet

- 假设有关系R(A, B)和S(B,C)，对二者进行自然连接操作
- 使用Map过程，把来自R的每个元组<a,b>转换成一个键值对<b, <R,a>>，其中的键就是属性B的值。把关系R包含到值中，这样做使得我们可以在Reduce阶段，只把那些来自R的元组和来自S的元组进行匹配。类似地，使用Map过程，把来自S的每个元组<b,c>，转换成一个键值对<b, <S,c>>
- 所有具有相同B值的元组被发送到同一个Reduce进程中，Reduce进程的任务是，把来自关系R和S的、具有相同属性B值的元组进行合并
- Reduce进程的输出则是连接后的元组<a,b,c>，输出被写到一个单独的输出文件中

用MapReduce实现关系的自然连接



10、wordcount代码（背一下!!!）

```
package com.ht.wordcount;
```

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```

import java.io.IOException;

public class WordCount {

    public static class WordCountMapper extends Mapper<LongWritable,Text,Text, IntWritable>{
        IntWritable intWritable = new IntWritable(1);
        @Override
        protected void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {
            // 1.读取数据
            String line = value.toString();
            // 2.切片
            String[] splits = line.split("\t");
            // 3.输出
            Text text = new Text();
            for (String split : splits) {
                text.set(split);
                context.write(text, intWritable);
            }
        }
    }

    public static class WordCountReducer extends Reducer<Text,IntWritable,Text,IntWritable>{
        @Override
        protected void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
            // 1.读取数据 <k,list<v1,v2,...,vn>>
            int sumVal = 0;
            for (IntWritable val:values){
                sumVal += val.get();
            }
            // 2.输出数据
            context.write(key,new IntWritable(sumVal));
        }
    }

    public static void main(String[] args) throws IOException, ClassNotFoundException,
    InterruptedException {
        // 1.hadoop运行信息
        Configuration configuration = new Configuration();
    }
}

```

```

// 2.获取hadoop实例
String jobName = "WordCount";
Job job = Job.getInstance(configuration, jobName);
// 3.设置程序的本地jar包
job.setJarByClass(WordCount.class);
// 4.关联mapper和reducer
job.setMapperClass(WordCountMapper.class);
job.setReducerClass(WordCountReducer.class);
// 5.设置mapper的输出kv
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(IntWritable.class);
// 6.设置reducer的输出kv（最终输出）
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
// 7.设置作业输入输出路径
Path inputPath = new Path("D:\\Document\\temp\\wordcount\\input.txt");
Path outputPath = new Path("D:\\Document\\temp\\wordcount\\output");
// 获取hdfs文件系统实例
FileSystem fileSystem = FileSystem.get(configuration);
if(fileSystem.exists(outputPath)){
    fileSystem.delete(outputPath,true);
}
// 8.设置输入输出格式
FileInputFormat.addInputPath(job,inputPath);
FileOutputFormat.setOutputPath(job, outputPath);
// 9.查看作业运行情况
System.out.println("job " + jobName + "is running...");
// 若成功打印1，不成功打印0
System.out.println(job.waitForCompletion(true) ? 1:0);
}
}

```

第8章 Hadoop再探讨

1、Hadoop的局限与不足

Hadoop1.0的核心组件（MapReduce和HDFS），主要存在以下不足：

- ① 抽象层次低，需人工编码

- ② 表达能力有限
- ③ 开发者自己管理作业（Job）之间的依赖关系
- ④ 难以看到程序整体逻辑
- ⑤ 执行迭代操作效率低
- ⑥ 资源浪费（Map和Reduce分两阶段执行）
- ⑦ 实时性差（适合批处理，不支持实时交互式）

2 针对Hadoop的改进与提升

Hadoop的优化与发展主要体现在两个方面：一方面是Hadoop自身两大核心组MapReduce和HDFS的架构设计改进；另一方面是Hadoop生态系统其它组件的不断丰富，加入了Pig、Tez、Spark和Kafka等新组件。

组件	Hadoop1.0的问题	Hadoop2.0的改进
HDFS	单一名称节点，存在单点失效问题	设计了HDFS HA，提供名称节点热备机制
HDFS	单一命名空间，无法实现资源隔离	设计了HDFS Federation，管理多个命名空间
MapReduce	资源管理效率低	设计了新的资源管理框架YARN
Pig	处理大规模数据的脚本语言，用户只需要编写几条简单的语句，系统会自动转换为MapReduce作业	抽象层次低，需要手工编写大量代码
Spark	基于内存的分布式并行编程框架，具有较高的实时性，并且较好支持迭代计算	延迟高，而且不适合执行迭代计算
Oozie	工作流和协作服务引擎，协调Hadoop上运行的不同任务	没有提供作业（Job）之间依赖关系管理机制，需要用户自己处理作业之间依赖关系
Tez	支持DAG作业的计算框架，对作业的操作进行重新分解和组合，形成一个大的DAG作业，减少不必要操作。	不同的MapReduce任务之间存在重复操作，降低了效率
Kafka	分布式发布订阅消息系统，一般作为企业大数据分析平台的数据交换枢纽，不同类型的分布式系统可以统一接入到Kafka，实现和Hadoop各个组件之间的不同类型数据的实时高效交换。	Hadoop生态系统中各个组件和其他产品之间缺乏统一的、高效的数据交换中介

3、HDFS HA

3.1、第二名称节点用途：

- ① 不是热备份。
- ② 主要是防止日志文件EditLog过大，导致名称节点失败恢复时消耗过多时间。
- ③ 附带起到冷备份功能。

3.2、HDFS HA 相关信息（如何工作？？？）

- 1).HDFS HA（高可用）是为了解决单点故障问题。
- 2).HA集群有两个名称节点，“活跃”和“待用”。
- 3).两个名称节点状态同步，可以借助与一个共享存储系统实现。
- 4).一旦活跃名称节点出故障，立刻切换到待命名称节点。
- 5).Zookeeper确保只有一个名称节点对外服务。
- 6).名称节点维护映射信息，数据节点同时向两个名称节点汇报信息

3.3、保持两个名称节点状态同步：

- 1) 可以借助共享存储系统
- 2) 需要给数据节点配置两个名称节点的地址，并把块的位置信息和心跳信息同时发送到这两个名称节点

4、HDFS Federation

4.1、HDFS Federation的设计

在HDFS Federation中，设计了多个相互独立的名称节点，使得HDFS的命名服务能够水平扩展，这些名称节点分别进行各自命名空间和块的管理，相互之间是联盟（Federation）关系，不需要彼此协调。HDFS Federation中，所有名称节点会共享底层的数据节点存储资源，数据节点向所有名称节点汇报。属于同一个命名空间的块构成一个“块池”。

4.2、HDFS Federation的访问方式

对于Federation中的多个命名空间，可以采用客户端挂载表方式进行数据共享和访问。客户可以访问不同的挂载点来访问不同的子命名空间。

4.3、为什么需要HDFS联邦，即它能解决什么问题？

HDFS联邦（Federation）能够解决单名称节点存在的以下几个问题：

- 1).HDFS集群扩展性** 多个名称节点各自分管一部分目录，使得集群可以扩展到更多节点，不再像HDFS1.0那样由于内存的限制制约了文件存储的数目。
- 2).性能更高效** 多个名称节点管理不同的数据，且同时对外提供服务，为用户提供更高的读写吞吐率。
- 3).良好的隔离性** 用户可根据需要将不同的业务数据交给不同的名称节点管理，这样不同业务之间的影响就很小。

需要注意的是，HDFS联邦不能解决单点故障问题，还需要为每一个名称节点部署一个后备名称节点。

5、新一代资源管理调度框架YARN

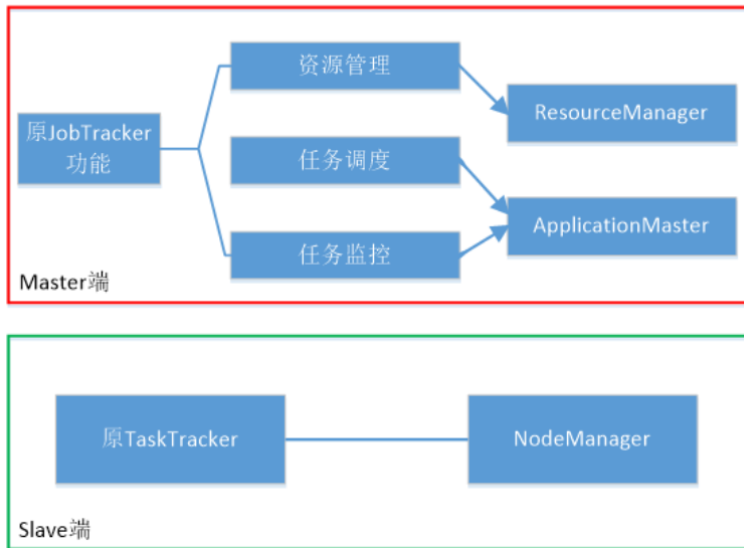
5.1、MapReduce1.0的缺陷：

- (1) 存在单点故障
- (2) JobTracker“大包大揽”导致任务过重（任务多时内存开销大，上限4000节点）。
- (3) 容易出现内存溢出（分配资源只考虑MapReduce任务数，不考虑CPU、内存的实际使用情况）
- (4) 资源划分不合理（强制划分为slot，包括Map slot和Reduce slot

5.2、YARN设计思路：**MapReduce1.0既是一个计算框架，也是一个资源管理调度框架**。到了Hadoop2.0以后，MapReduce1.0中的资源管理调度功能，被单独分离出来形成了YARN，它是一个**纯粹的资源管理调度框架**，而不是一个计算框架。被剥离了资源管理调度功能的MapReduce 框架就变成了**MapReduce2.0**，它是运行在YARN之上的一个纯粹的计算框架

5.3、描述YARN架构中各个组件的功能

YARN架构思路：将原JobTracker三大功能拆分



ResourceManager

- 1).处理客户端请求
- 2).启动/监控ApplicationMaster
- 3).监控NodeManager
- 4).资源分配与调度

ApplicationMaster

- 1).为应用程序申请资源，并分配给内部任务
- 2).任务调度、监控与容错

NodeManager

- 1).单个节点上的资源管理
- 2).处理来自ResourceManger的命令
- 3).处理来自ApplicationMaster的命令

ResourceManager (RM) 是一个全局的资源管理器，负责整个系统的资源管理和分配，主要包括两个组件，即调度器 (Scheduler) 和应用程序管理器 (Applications Manager)。容器 (Container) 作为动态资源分配单位

5.4、YARN工作流程

步骤1：用户编写客户端应用程序，向YARN提交应用程序，提交的内容包括ApplicationMaster程序、启动ApplicationMaster的命令、用户程序等。

步骤2：YARN中的ResourceManager负责接收和处理来自客户端的请求，为应用程序分配一个容器，在该容器中启动一个ApplicationMaster。

步骤3：ApplicationMaster被创建后会首先向ResourceManager注册。

步骤4：ApplicationMaster采用轮询的方式向ResourceManager申请资源。

步骤5：ResourceManager以“容器”的形式向提出申请的ApplicationMaster分配资源。

步骤6：在容器中启动任务（运行环境、脚本）。

步骤7：各个任务向ApplicationMaster汇报自己的状态和进度。

步骤8：应用程序运行完成后，ApplicationMaster向ResourceManager的应用程序管理器注销并关闭自己。

YARN相对于MapReduce1.0来说具有以下优势：

大大减少了承担中心服务功能的ResourceManager的资源消耗。

ApplicationMaster来完成需要大量资源消耗的任务调度和监控。

多个作业对应多个ApplicationMaster，实现了监控分布化。

YARN中的资源管理比MapReduce1.0更加高效：以容器为单位，而不是以slot为单位。

YARN的目标就是实现“一个集群多个框架”，为什么？

- 一个企业当中同时存在各种不同的业务应用场景，需要采用不同的计算框架
- MapReduce实现离线批处理
- 使用Impala实现实时交互式查询分析
- 使用Storm实现流式数据实时分析
- 使用Spark实现迭代计算

为了避免不同类型应用之间互相干扰，企业就需要把内部的服务器拆分成多个集群，分别安装运行不同的计算框架，即“一个框架一个集群”

6、简述Pig、Tez、Kafka的功能

Pig：

- 提供了类似SQL的Pig Latin语言
- 允许用户通过编写简单的脚本来实现复杂的数据分析，而不需要编写复杂的MapReduce应用程序
- 用户在编写Pig程序的时候，不需要关心程序的运行效率，这就大大减少了用户编程时间
- 通过配合使用Pig和Hadoop，在处理海量数据时就可以实现事半功倍的效果，难度要小很多，并且用更少的代码量实现了相同的数据处理分析功能

Tez：

- Tez是Apache开源的支持DAG作业的计算框架，它直接源于MapReduce框架
- 核心思想是将Map和Reduce两个操作进一步拆分
- 分解后的元操作可以任意灵活组合，产生新的操作
- 通过DAG作业的方式运行MapReduce作业，提供了程序运行的整体处理逻辑，就可以去除工作流当中多余的Map阶段，减少不必要的操作，提升数据处理的性能

Kafka：

- Kafka是一种高吞吐量的分布式发布订阅消息系统，用户通过Kafka系统可以发布大量的消息，同时也能实时订阅消费消息
- Kafka可以同时满足在线实时处理和批量离线处理

第9章 数据仓库Hive

数据仓库的体系结构通常包含4个层次：数据源、数据存储和管理、数据服务、数据应用。

Hive是一个构建在Hadoop之上的数据仓库工具，Hive在某种程度上可以看作用户编程接口，其本身并不存储和处理数据，而是依赖HDFS来存储数据，依赖MapReduce（或者Tez、Spark）来处理数据。Hive定义了简单的类似SQL的查询语言----HiveQL，它与大部分SQL语法兼容。

1、Hive把HiveQL语句转换成MapReduce任务后，采用**批处理**的方式对海量数据进行处理。数据仓库存储的是**静态数据**，构建于数据仓库上的应用程序只进行相关的静态数据分析

2、Hive本身提供了一系列对数据进行抽取、转换、加载的工具，**可以存储、查询和分析存储在Hadoop中的大规模数据**。

Hive可以用来进行数据提取转化加载（ETL）

Hive定义了简单的类似SQL的查询语句，称为HiveQL（HQL），它允许熟悉SQL的用户查询数据。Pig可以作为Hive的替代工具，它是一种数据流语言和运行环境，适用于在Hadoop平台上查询半结构化数据集，常用于ETL过程的一部分

在Hadoop之上设计Hive，是为了减少复杂MapReduce应用程序的编写工作，在Hadoop上设计HBase则是为了实现对数据的实时访问

Hive 的执行延迟比较高，因此 Hive 常用于数据分析，对实时性要求不高的场合。**Hive具有SQL数据库的外表，但应用场景完全不同，Hive只适合用来做批量海量数据统计分析，也就是数据仓库。**

所有 Hive 的数据都是存储在 HDFS 中的，Hive 中不建议对数据的改写，Hive 要访问数据中满足条件的特定值时，需要暴力扫描整个数据，因此访问延迟较高，有对数据中的某些 Key 建立索引。由于 Hive 是建立在 Hadoop 之上的，因此 Hive 的可扩展性是和 Hadoop 的可扩展性是一致的

Hive主要由以下3个模块组成，：**用户接口模块、驱动模块以及元数据存储模块。**

1、用户接口模块

用户接口模块包括CLI（命令行客户端工具）、Hive网页接口（Hive Web Interface, HWI）、JDBC、ODBC、Thrift Server等，用来实现外部应用对Hive的访问。

2、驱动模块（Driver）

驱动模块（Driver）包括解析器，编译器、优化器、执行器等，驱动模块负责把HiveQL语句转换成一系列MapReduce作业

3、元数据存储模块（Metastore）

元数据存储模块（Metastore）是一个独立的关系数据库。Hive的元数据存储于关系数据库中

Impala

由于Hive采用MapReduce来完成批量数据处理，因此，实时性不好，查询延迟较高。Impala作为开源大数据分析引擎，**支持实时计算**，它提供了与Hive类似的功能，并在性能上比Hive高出3~30倍。Impala最初是参照Dremel系统进行设计的。

Impala和Hive、HDFS、HBase等工具是统一部署在一个Hadoop平台上的。Impala主要由Impalad、State Store和CLI三部分组成。

Impala与Hive的比较

（1）相同点

- 1) Hive与Impala使用相同的存储数据池，都支持把数据存储于HDFS和HBase中
- 2) Hive与Impala使用相同的元数据。
- 3) Hive与Impala中对SQL的解释处理比较相似，都是通过词法与分析生成执行计划。

（2）不同点

- 1) **Hive适合进行长时间的批处理查询分析，Impala适合进行实时交互式查询。**
- 2) 当采用MapReduce作为执行引擎时，Hive依赖于MapReduce计算框架，执行计划组合成管道型的MapReduce任务模式进行执行，Impala则把执行计划表现为一棵完整的执行计划树，可以更自然地分发执行计划到各个Impalad执行查询。
- 3) Hive在执行过程中，如果内存放不下所有数据，则会使用外存，以保证查询能顺序执行完成，而Impala在遇到内存放不下数据时，不会利用外存。所以，Impala目前处理查询时会受到一定的限制。**使得Impala更适合处理输出数据较小的查询请求，而对于大数据量的批处理，Hive依然是更好的选择。**事实上，把Hive与Impala配合使用效果最佳，可以先使用Hive进行数据转换处理，再使用Impala对Hive处理后的结果数据集进行快速的数据分析。

第10章 Spark

Spark基于内存计算的大数据并行计算框架，可用于构建大型的、低延迟的数据分析应用程序。

Spark使用先进的有向无环图（Directed Acyclic Graph,DAG）执行引擎，以支持循环数据流与内存计算

Scala是一门现代的多范式编程语言，平滑地集成了面向面向对象和函数式语言的特性。Scala是Spark的主要编程语言，但Spark还支持Java、Python、R作为编程语言。**Spark Shell中可进行交互式编程**

	Hadoop	Spark
类型	分布式基础平台，包含计算、存储、调度	分布式计算工具
场景	大规模数据集上的批处理	迭代计算、交互式计算、流计算

Spark可以用于批处理、交互式查询(Spark SQL)、实时流处理(Spark Streaming)、机器学习(Spark MLlib)和图计算(GraphX)

Spark生态系统

应用场景	时间跨度	其他框架	Spark生态系统中的组件
复杂的批量数据处理	小时级	MapReduce、Hive	Spark Core
基于历史数据的交互式查询	分钟级、秒级	Impala、Dremel、Drill	Spark SQL
基于实时数据流的数据处理	毫秒级、秒级	Storm、S4	Spark Streaming Structured Streaming
基于历史数据的数据挖掘	--	Mahout	MLlib
图结构数据的处理	--	Pregel、Hama	GraphX

- Spark将数据载入内存后，之后的迭代计算都可以直接使用内存中的中间结果作运算，避免了从磁盘中频繁读取数据
- Spark基于DAG的任务调度执行机制，要优于Hadoop MapReduce的迭代执行机制

spark设计理念：一个软件栈满足不同应用场景

- RDD：是Resilient Distributed Dataset（弹性分布式数据集）的简称，是分布式内存的一个抽象概念，提供了一种高度受限的共享内存模型
- DAG：是Directed Acyclic Graph（有向无环图）的简称，反映RDD之间的依赖关系

第11章 流计算

1.流计算与批处理的区别

批处理：处理离线数据。单个处理数据量大，处理速度比流慢。

流计算：处理实时产生的数据。单次处理的数据量小，但处理速度更快。

2、流处理系统与传统的数据处理系统有如下不同

流处理系统处理的是实时的数据，而传统的数据处理系统处理的是预先存储好的静态数据。

用户通过流处理系统获取的是实时结果，而通过传统的数据处理系统，获取的是过去某一时刻的结果。

流处理系统无需用户主动发出查询，实时查询服务可以主动将实时结果推送给用户。

3、流计算框架Storm，Spark Streaming，Samza

第12章 Flink

Flink的主要特性包括：批流一体化、精密的状态管理、事件时间支持以及精确一次（Exactly-once）的状态一致性保障等

流处理架构的要求：低延迟、高吞吐、高性能

Strom：能做到低延迟，无法实现高吞吐，也不能在故障发生时准确地处理计算状态。

Spark Streaming：通过采用微批处理方法实现了高吞吐和容错性，但无法实现低延迟

Flink：实现了Google Dataflow流计算模型，兼顾高吞吐、低延迟和高性能，并同时支持批处理和流处理。此外，Flink支持高度容错的状态管理。

Flink核心组件栈分为 三层：物理部署层、Runtime核心层和API&Libraries层。

第13章 图计算

图数据结构很好地表达了数据之间的关联性

传统的图计算算法存在的典型问题：

- 1、尝尝表现出较差的内存访问局部性
- 2、针对单个顶点的处理工作过少
- 3、计算过程中伴随着并行度的改变

通用的图计算软件：

- 1、基于遍历算法，实时的图数据库：Neo4j, OrientDB, DEX, Infinite Graph
- 2、以图顶点为中心，基于消息传递批处理的并行引擎：Golden, Giraph, Pregel, Hama

Pregel是谷歌公司发布的一款商业图计算产品，基于BSP模型实现的并行图处理系统，采用了消息传递模型，一次BSP计算过程包括一系列全局超步（所谓的超步就是计算中的一次迭代）

局部计算：每个参与的处理器的都有自身的计算任务，

通讯：处理器群相互交换数据

栅栏同步(Barrier Synchronization)：当一个处理器遇到“路障”（或栅栏），会等到其他所有处理器完成它们的计算步骤

“三驾马车”：

Caffeine：大规模网页索引构建

Dremel：实时交互

Pregel：基于BSP并行图计算处理

计算模型作为有向图作为输入

BSP：整体同步并行计算模型

第16.4章 基于大数据的综合健康服务平台

大数据在采集、存储、处理等环节所独有的特征与我们目前处理数据的方法存在着较大的不同。医疗健康大数据将辅助医疗服务从疾病诊疗转变为疾病预防和居民健康管理、从单一的医疗服务转变为医疗公共机构的综合管理服务、从基于经验的医疗服务转变为基于循证医学的个性化医疗服务。本文将结合我国医疗健康信息化发展现状及其数据自身特点，以多源异构的海量医疗健康数据为基础，设计研究基于大数据的医疗健康信息化服务平台。

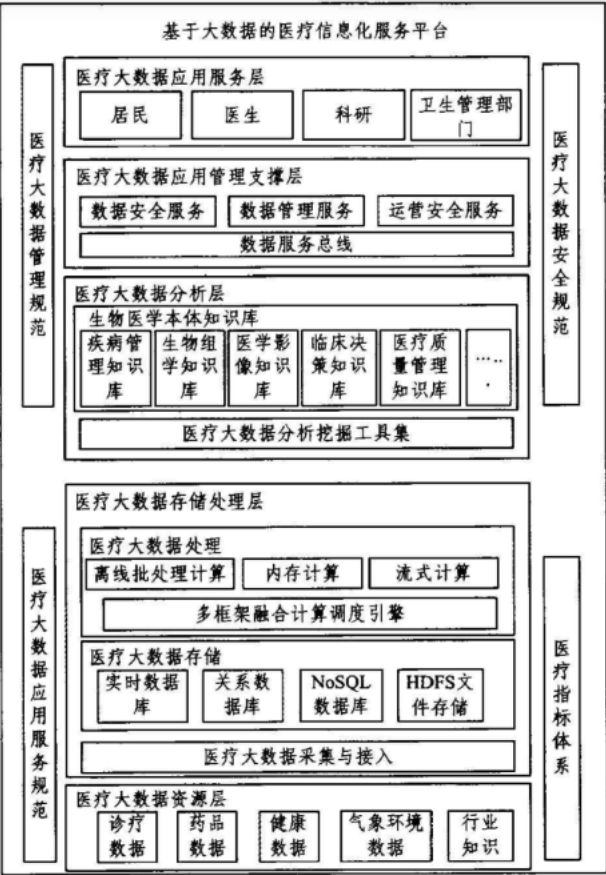


图1 基于大数据的医疗信息化服务平台架构

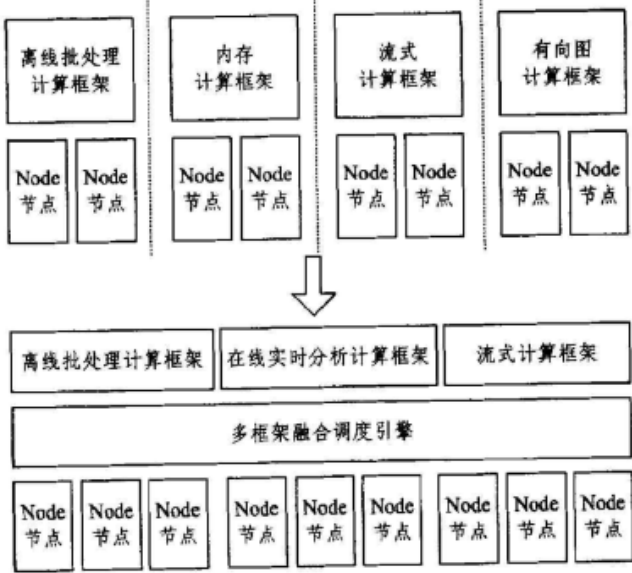


图3 多框架融合管理引擎

多框架融合管理引擎通过对集群资源的统一管理，将CPU、内存通过虚拟化手段形成资源池，所有计算框架对资源的需求都需要通过向多框架融合管理引擎申请而获得，不同用户申请的资源都是逻辑

构建医疗大数据资源层：研究对多源异构数据源的对接技术，研发多源异构数据源采集平台，通过融合处理来构建覆盖诊疗数据、药品数据、健康数据、气象环境数据、行业知识的医疗大数据资源层。

研发医疗大数据存储处理层：研发针对大数据采集、处理、存储的医疗健康大数据存储处理层。采用分布式计算框架，实现对不同计算框架的统一资源调度管理技术和引擎。

研发医疗大数据分析层：在存储处理层的基础上，分析层将构建医疗健康大数据的挖掘分析工具及知识库。

研发医疗大数据应用支撑层：应用支撑层将为对外的标准化服务如注册、封装、调用、开发提供医疗大数据应用支撑管理平台，为相互逻辑隔离、独立运行的数据提供方、数据使用方创建一个交互的环境。

研发医疗大数据应用层：开展面向居民、公共卫生、医生、科研人员和医疗管理机构的一系列应用服务。

一、医疗大数据资源层

1, 诊疗数据: 诊疗数据包括门诊诊断数据, 住院数据、处方数据, 检验检查报告等, 这类数据主要来源于区域医疗系统、社区医疗系统、医疗机构系统等, 其大多是结构化的, 但在某些数据字段 (如出院小结、诊断说明等) 是非结构化的文本信息,

2 药品数据: 药品数据来源于公开的医药监管部门, 数据通常是结构化、局部集结构化的, 数据量大约为数GB, 通

常需要保存很长时间。

3.健康数据: 其通常是流式地更新数据, 并且数据已经高度的结构化,

4, 医疗知识库: 医疗知识库主要来源于权威数据源的专业公共知识库, 或者通过对医疗大数据分析而建立起来的自有知识岸

二、医疗大数据存储处理层

针对不同的数据源、不同的数据格式、不同的数据逻辑关系, 医疗健康大数据存储处理平台提供实时数据库、关系数据库、NoSQL数据存储、HDFS文件存储等多种专用的存储服务和系统, 为数据的高效存储和有效管理提供保障。

医疗信息通常由区域医疗、社区医疗系统提供, 是格式化数据, 其数据每口更新; 健康监护数据需要面对海量的并发监护采集数据的提交, 数据量较小但数量巨大且并发, 需要快速, 及时的处理; 影像数据的数据量巨大, 但数量相对较少, 不同的数据需要采用不同的处理方法, 以提高处理的效率。该平台构建了多框架融合计算调度引擎, 在其之上分别提供离线批处理计算、在线实时分析计算、流式计算等多种计算框架的集成调用。

隔离的。所有被申请的资源都会受到多框架融合管理引擎的监管, 当资源失效或者负载过高时, 其可以对资源进行动态的分配和调整, 以提高利用效率。

1. 离线批处理计算方式 (Map-Reduce)

采用离线批处理计算框架, 通过 Map、Reduce 操作, 可以提供对医疗和健康数据的离线批处理分析服务。

2. 在线实时分析计算框架 (Spark)

在线实时分析计算框架采用了内存分布数据集分析技术, 结合平台提供的实时数据库, 提供了交互式查询服务。

在线实时分析计算框架引进了内存集群计算概念, 在内存集群计算中将数据集缓存在内存中, 以缩短访问延迟。

在线实时分析计算框架同时引进了名为弹性分布式数据集 (RDD) 的抽象。RDD 是分布在一组节点中的只读对象集合。这些集合是弹性的, 如果数据集丢失, 其可以对它们进行重建。

3. 流式计算框架 (Storm)

从数据源特征角度来看, 流式计算框架与 Map-Reduce 明显的不同是流式计算框架的数据源是动态的, 即收到一条便处理一条, 面对不断更新的医疗大数据, 流式计算框架可以快速、高效地处理相关的数据。

3.3 医疗大数据分析层

在医疗大数据分析层, 将着重解决两个层面的分析工作: 1) 面向医疗大数据分析的分析挖掘, 将传统的通用数据挖掘工具进行优化改造及并行化实现, 在医疗领域本体的支持下, 为医疗大数据应用服务提供专用的分析模型库; 2) 在对医疗大数据挖掘利用的基础上, 辅以领域知识构建技术, 建立生物学本体知识库模型。

1. 面向医疗大数据的分析挖掘

1) 采用特异群组挖掘、热点识别模型、多标记分类、直推式分类、效用序列模式挖掘、相关性分析、时序演变分析、比例失调分析和通用医学统计分析算法分析医疗大数据, 在不牺牲挖掘效率和挖掘质量的情况下通过分布式计算等技术手段, 从算法并行优化的角度提高计算效率。

2) 采用布隆过滤器 (Bloom filter)、哈希 (Hashing) 等技术, 解决算法优化时可能出现的数据维度灾难问题, 实现数据的快速查找和比对, 降低计算内消消耗。

Zookeeper三种部署模式：单机模式（单台服务器），集群模式（多台服务器），伪集群模式（一台服务器部署多个Zookeeper服务器）